HW6

STAT 651

RJ Cass

2026-03-09

1

Verify that $X_t \sim N(0, (1-\phi^2)^{-1})$ and $X_{t+1}|X_t \sim N(\phi X_t, 1)$, with $-1 < \phi < 1$ satisfies detailed balance.

$$
\begin{aligned}
\pi(X_t)k(X_{t+1}|X_t) &= \pi(X_{t+1})k(X_t|X_{t+1}) \\
[2\pi(1-\phi^2)^{-1}]^{-1/2}e^{-1/2(1-\phi^2)X_t^2}(2\pi)^{-1/2}e^{-1/2(X_{t+1}-\phi X_t)^2} &= [2\pi(1-\phi^2)^{-1}]^{-1/2}e^{-1/2(1-\phi^2)X_{t+1}^2} \\
&\quad (2\pi)^{-1/2}e^{-1/2(X_t-\phi X_{t+1})^2} \\
e^{-1/2(1-\phi^2)X_t^2}e^{-1/2(X_{t+1}-\phi X_t)^2} &= e^{-1/2(1-\phi^2)X_{t+1}^2}e^{-1/2(X_t-\phi X_{t+1})^2} \\
(1-\phi^2)X_t^2 + (X_{t+1}-\phi X_t)^2 &= (1-\phi^2)X_{t+1}^2 + (X_t-\phi X_{t+1})^2 \\
X_t^2 - \phi^2 X_t^2 + X_{t+1}^2 - 2\phi X_t X_{t+1} + \phi^2 X_t^2 &= X_{t+1}^2 - \phi^2 X_{t+1}^2 + X_t^2 - 2\phi X_{t+1}X_t + \phi^2 X_{t+1}^2 \\
X_t^2 + X_{t+1}^2 - 2\phi X_t X_{t+1} &= X_t^2 + X_{t+1}^2 - 2\phi X_t X_{t+1} \\
0 &= 0
\end{aligned}
$$

Thus it satisifies detailed balance.

2

For this question, I provide the plot outputs within each question. After part (e) I have a table which shows the process times and posterior means for each method for comparison

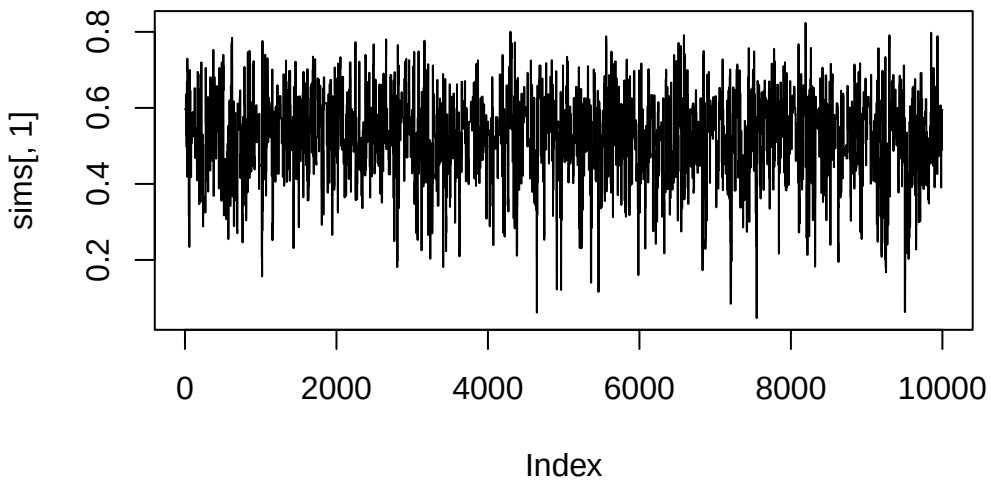
a)

Gibbs sampler using a pair of univariate random-walk Metropolis-Hastings samplers to update each parameter given the other.

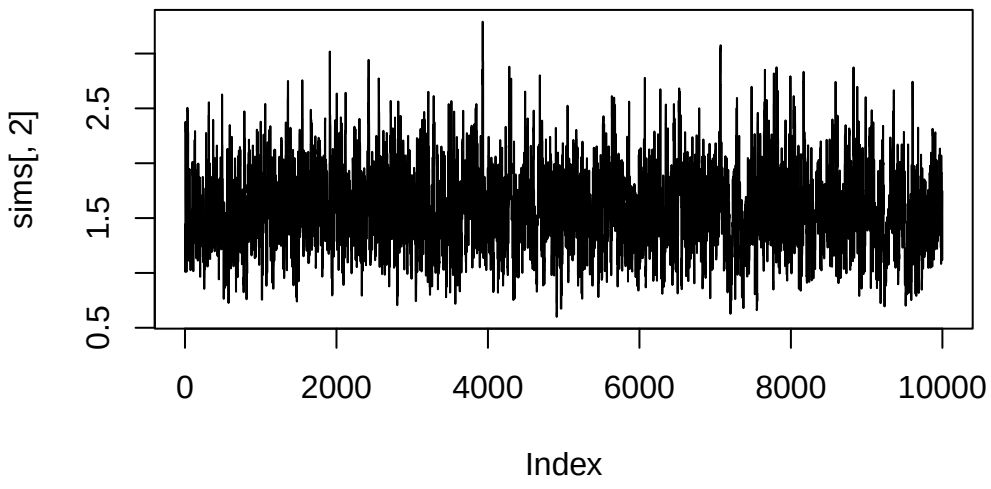
```r
set.seed(1337)
y <- c(rep(0, 25), rep(1, 6), rep(2, 4), rep(3, 3), 5)
n <- length(y)
n0 <- sum(y == 0)
sumy <- sum(y)

initial_pi <- .5
initial_lambda <- 1.5
```

```r
log_lik <- function(p, lambda, x) {
  # Log-likelihood components
  loglik_zero <- log(p + (1 - p) * exp(-lambda))
  loglik_nonzero <- log(1 - p) - lambda + x * log(lambda) - lgamma(x + 1)

  # Combine
  sum(ifelse(x == 0, loglik_zero, loglik_nonzero))
}

logfc_pi <- function(p, lambda, x) {
  log_lik(p, lambda, x) + dbeta(p, 1, 1, log = TRUE)
}

logfc_lambda <- function(p, lambda, x) {
  log_lik(p, lambda, x) + dgamma(lambda, 2, 1, log = TRUE)
}

mh_pi <- function(p, lambda, x) {
    proposal <- rnorm(1, mean = p, sd = .5)
    # Pi is bounded by 0 and 1
    if ((proposal >= 1) | (proposal <= 0)) {return(p)}
    odds <- logfc_pi(proposal, lambda, x) - logfc_pi(p, lambda, x)
    ifelse(log(runif(1)) <= odds, proposal, p)
}

mh_lambda <- function(p, lambda, x) {
    proposal <- rnorm(1, mean = lambda, sd = .5)
    # Lambda is lower bounded by 0
    if (proposal <= 0) {return(lambda)}
    odds <- logfc_lambda(p, proposal, x) - logfc_lambda(p, lambda, x)
    ifelse(log(runif(1)) <= odds, proposal, lambda)
}

# Initialize
state <- list(pi = initial_pi, lambda = initial_lambda)
n_iter <- 10000
sims <- as.data.frame(matrix(NA, nrow = n_iter, ncol = length(state)))
colnames(sims) <- names(state)
```

```
start_time <- Sys.time()
for (i in 1:n_iter) {
    # Pi is bounded 0 to 1
    state$pi <- mh_pi(state$pi, state$lambda, y)
    state$lambda <- mh_lambda(state$pi, state$lambda, y)

    # Save current state
    sims[i, 'pi'] <- state$pi
    sims[i, 'lambda'] <- state$lambda
}
end_time <- Sys.time()
run_time_a <- round(end_time - start_time, 4)

plot(sims[, 1], type = 'l')
```




```
plot(sims[, 2], type = 'l')
```

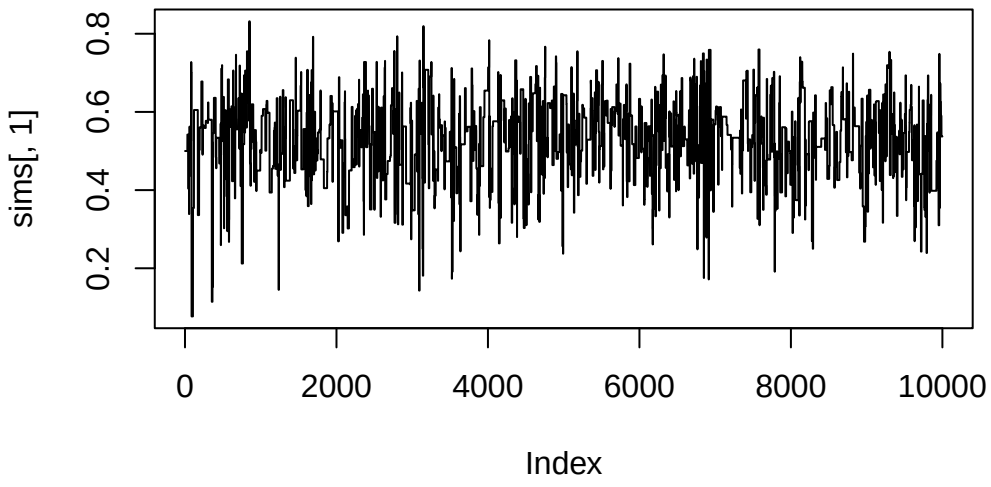


```r
pi_post_mean_a <- round(mean(sims[, 1]), 4)
lambda_post_mean_a <- round(mean(sims[, 2]), 4)
```

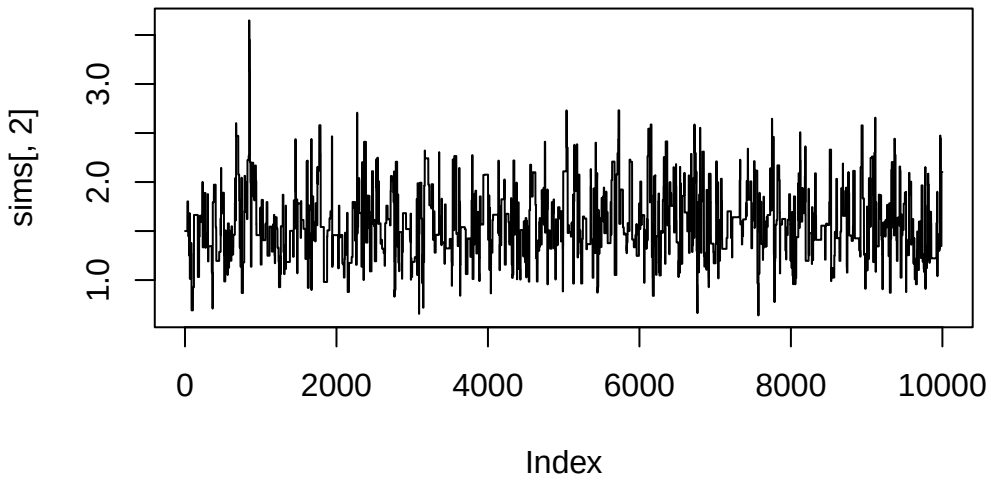
b)

```r
suppressPackageStartupMessages(library("MASS"))

# Full Conditional is the Joint Posterior
# Joint posterior given in solutions for HW4 5e
log_joint_post <- function(p, lambda){
    n0*log(p + (1 - p)*exp(-lambda)) + (n-n0)*log(1 - p) + (sumy + 1)*log(lambda) - ((n-n0) + 1
}

mh <- function(p, lambda) {
    # Sample from multivaraite normal, use Joint Posterior to evaluate odds for acceptance rati
    # Variance matrix, just guessing values
    Sigma <- matrix(c(.5, 0, 0, .5), nrow = 2, ncol = 2)
    mu <- c(p, lambda)
    proposal <- mvrnorm(1, mu, Sigma)
    # Pi is bounded by 0 and 1, Lambda lower bound by 0
    if ((proposal[1] >= 1) | (proposal[1] <= 0) | (proposal[2] <= 0)) {return(c(p, lambda))}
    odds <- log_joint_post(proposal[1], proposal[2]) - log_joint_post(p, lambda)
    ifelse(log(runif(1)) <= odds, return(proposal), return(c(p, lambda)))
}

# Initialize
state <- list(pi = initial_pi, lambda = initial_lambda)
n_iter <- 10000
sims <- as.data.frame(matrix(NA, nrow = n_iter, ncol = length(state)))
colnames(sims) <- names(state)

start_time <- Sys.time()
for (i in 1:n_iter) {
    # Pi is bounded 0 to 1
    res <- mh(state$pi, state$lambda)
    state$pi <- res[1]
    state$lambda <- res[2]

    # Save current state
    sims[i, 'pi'] <- state$pi
    sims[i, 'lambda'] <- state$lambda
}
end_time <- Sys.time()
run_time_b <- round(end_time - start_time, 4)
```

```r
plot(sims[, 1], type = 'l')
```




```r
plot(sims[, 2], type = 'l')
```

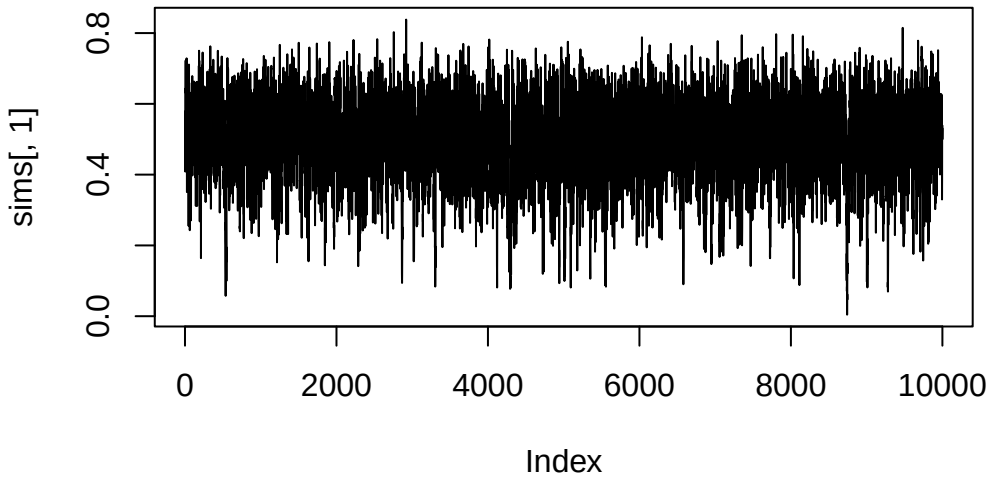



```r
pi_post_mean_b <- round(mean(sims[, 1]), 4)
lambda_post_mean_b <- round(mean(sims[, 2]), 4)
```

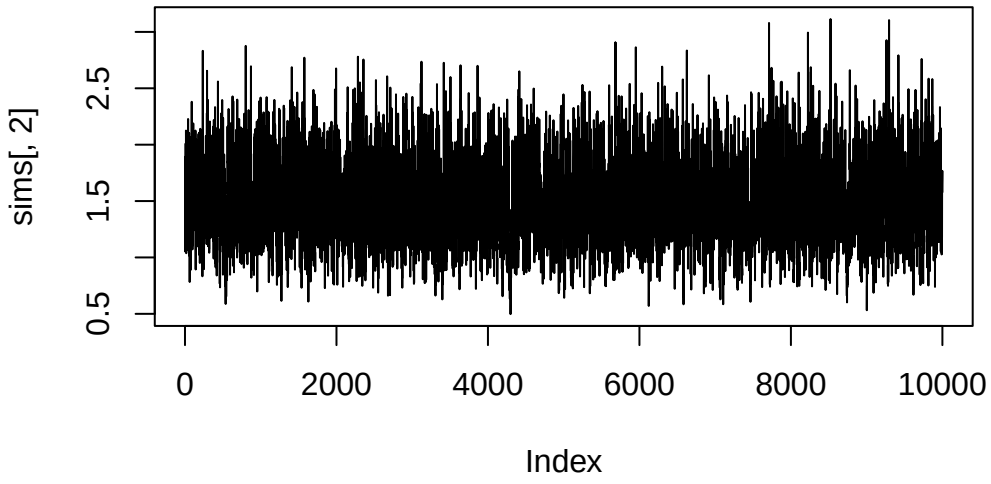
c)

```r
# Full Conditionals for the parameters were derived in class
update_pi <- function(alpha, sumz, beta, n) {
    rbeta(1, alpha + sumz, beta + n - sumz)
}

update_lambda <- function(k, sumy, theta, n, sumz) {
    rgamma(1, k + sumy, theta + n - sumz)
```

```r
}

sample_z <- function(w1, w2) {
    sample(c(1, 0), size = 1, prob = c(w1, w2))
}

update_z <- function(p, lambda, y) {
    w1 <- p*(y == 0)
    w2 <- sapply(y, \(x) exp(-lambda) * lambda^x / factorial(x) * (1-p))
    mapply(sample_z, w1, w2)
}

# Initialize
state <- list(pi = initial_pi, lambda = initial_lambda)
alpha <- 1
beta <- 2
k <- 3
theta <- 2
n_iter <- 10000
sims <- as.data.frame(matrix(NA, nrow = n_iter, ncol = length(state)))
colnames(sims) <- names(state)

start_time <- Sys.time()
for (i in 1:n_iter) {
    z <- update_z(state$p, state$lambda, y)
    state$pi <- update_pi(alpha, sum(z), beta, n)
    state$lambda <- update_lambda(k, sumy, theta, n, sum(z))

    # Save current state
    sims[i, 'pi'] <- state$pi
    sims[i, 'lambda'] <- state$lambda
}
end_time <- Sys.time()
run_time_c <- round(end_time - start_time, 4)

plot(sims[, 1], type = 'l')
```




```
plot(sims[, 2], type = 'l')
```

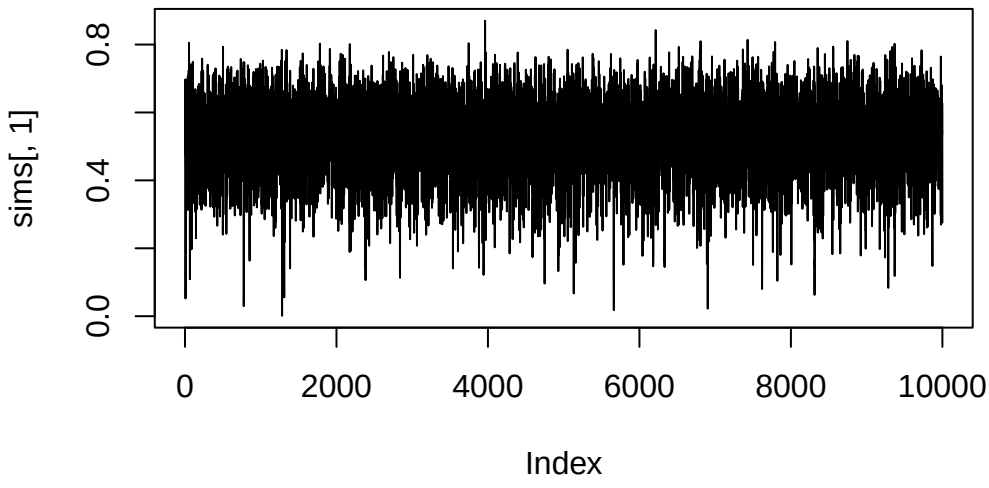



```
pi_post_mean_c <- round(mean(sims[, 1]), 4)
lambda_post_mean_c <- round(mean(sims[, 2]), 4)
```

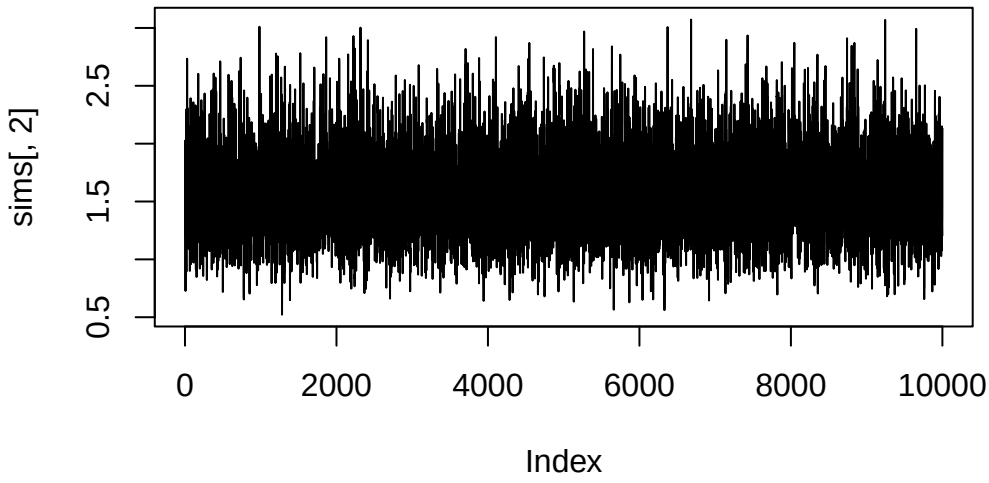
d)

```
suppressPackageStartupMessages(library("qslice"))

# Initialize
state <- list(pi = initial_pi, lambda = initial_lambda)
n_iter <- 10000
sims <- as.data.frame(matrix(NA, nrow = n_iter, ncol = length(state)))
colnames(sims) <- names(state)

start_time <- Sys.time()
for (i in 1:n_iter) {
```

```
    # Update Pi
    logtarget <- function(p) {
        logfc_pi(p, state$lambda, y)
    }
    tmp <- slice_stepping_out(state$pi, log_target = logtarget, w = .1)
    state$pi <- tmp$x

    # Update Lambda
    logtarget <- function(lambda) {
        logfc_lambda(state$pi, lambda, y)
    }
    tmp <- slice_stepping_out(state$lambda, log_target = logtarget, w = .1)
    state$lambda <- tmp$x

    # Save current state
    sims[i, 'pi'] <- state$pi
    sims[i, 'lambda'] <- state$lambda
}

end_time <- Sys.time()
run_time_d <- round(end_time - start_time, 4)

plot(sims[, 1], type = 'l')
```




```
plot(sims[, 2], type = 'l')
```

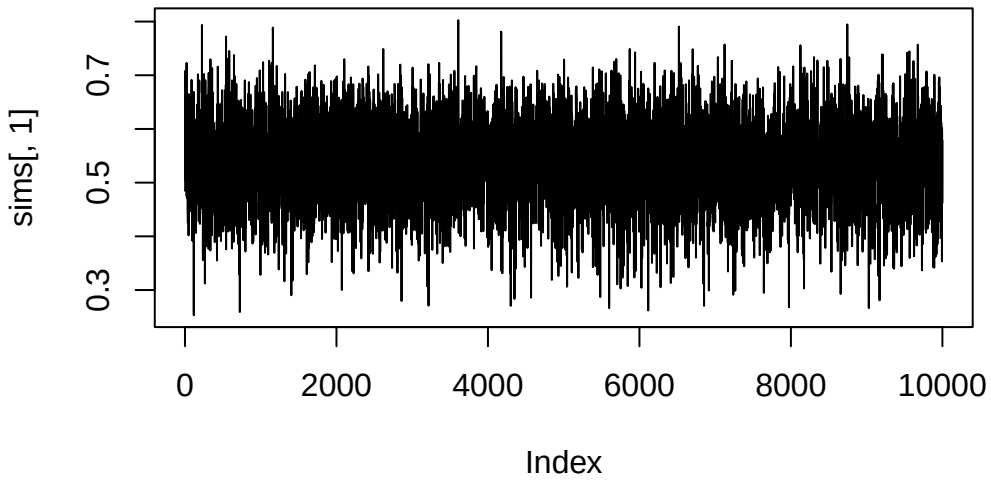



```
pi_post_mean_d <- round(mean(sims[, 1]), 4)
lambda_post_mean_d <- round(mean(sims[, 2]), 4)
```

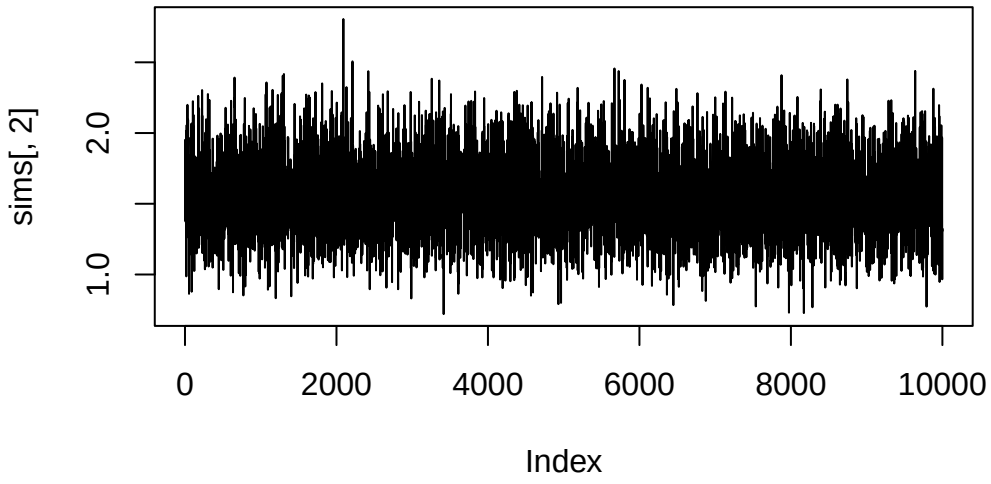
e)

```
mh <- function(p, lambda) {
    # Sample from multivaraite normal, use Joint Posterior to evaluate odds for acceptance rati
    # Values for Mu and Sigma matrices given in the HW5 1 solution
    Sigma <- matrix(c(.01165822, .01692337, .01692337, .13601894), nrow = 2, ncol = 2)
    mu <- c(.5432177, 1.5415007)
    proposal <- mvrnorm(1, mu, Sigma)
    # Pi is bounded by 0 and 1, Lambda lower bound by 0
    if ((proposal[1] >= 1) | (proposal[1] <= 0) | (proposal[2] <= 0)) {return(c(p, lambda))}
    odds <- log_joint_post(proposal[1], proposal[2]) - log_joint_post(p, lambda)
    ifelse(log(runif(1)) <= odds, return(proposal), return(c(p, lambda)))
}


# Initialize
state <- list(pi = initial_pi, lambda = initial_lambda)
n_iter <- 10000
sims <- as.data.frame(matrix(NA, nrow = n_iter, ncol = length(state)))
colnames(sims) <- names(state)

start_time <- Sys.time()
for (i in 1:n_iter) {
    res <- mh(state$pi, state$lambda)
    state$pi <- res[1]
    state$lambda <- res[2]

    # Save current state
```

```
    sims[i, 'pi'] <- state$pi
    sims[i, 'lambda'] <- state$lambda
}

end_time <- Sys.time()
run_time_e <- round(end_time - start_time, 4)

plot(sims[, 1], type = 'l')
```




```
plot(sims[, 2], type = 'l')
```




```
pi_post_mean_e <- round(mean(sims[, 1]), 4)
lambda_post_mean_e <- round(mean(sims[, 2]), 4)
```

Comparison

Part	CPU Time	Post. Mean. π	Post. Mean λ
A	0.5514	0.5228	1.574
B	0.5046	0.5309	1.5848
C	1.6266	0.4954	1.5004
D	1.6436	0.5243	1.5751
E	0.479	0.5392	1.5544

As shown in the table, they all are within the ballparks for the same values for the posterior means (all estimates are within .05 for π, and within .1 for λ). However, computation times for C and D are clearly more significant than the other methods. C makes sense because every iteration it has to iterate over the full array of y to get the z's, and as the length of y increases, the difference in CPU time for that method would increase a lot. D I guess makes sense, depending on the methods used in that slice_stepping_out function. Besides that, the bivariate MH methods were fastest, which makes sense as they just do 1 sample, so a bit less work per iteration. I did test with different initial values for π and λ, and interestingly, method C always gave a value below .5.

Intellectually, the normal approximation felt like probably the most complex in just finding the actual values for μ and Σ, but once that is done it's easy. However, it does not perfrom any better than the other version doing the same method, which was setup just using set values I picked out of thin air. That surprised me. Conceptually, the Gibbs samplers make the most sense and are probably the easiest to code, so that's nice, but sad they take longer.

3

$f(y_i|\mu,\sigma^2) \sim N(\mu,\sigma^2), \pi(\mu) \sim N(m,v), \pi(\sigma) \sim Cauchy^+(s_0)$

As shown in class:

$p(y,\mu,\sigma) \propto \prod_{i=1}^{n}\left[(2\pi\sigma^2)^{-1/2}e^{\frac{-1}{2\sigma^2}(y_i-\mu)^2}\right](2\pi v)^{-1/2}e^{\frac{-1}{2v^2}(\mu-m)^2}\frac{1}{s_0(1+(\frac{\sigma}{s_0})^2)}$

I'm going to pick my priors based on our previous experience with statistics exams, and pick $m = 80, v = 7, s_0 = 3$. This is saying that I expect μ to be centered on 80, but I don't have a lot of confidence in that, so I'm saying it make take a wide range of values. I honestly don't understand the Half-cauchy parameter very well, as I understand it the parameter is basically the median, so I'm just setting it to some middle-ish value of what I think the standard deviation might be.

```
dt <- read.table("stat-exam.dat", header = FALSE, sep = "")
y <- as.numeric(dt[, 1])
sumy <- sum(y)
ybar <- mean(y)
n <- length(y)
m <- 80
v <- 7
s0 <- 3
initial_mu <- 80
initial_sigma <- 5
```

Simiar to the previous problem, for each question I will provide the graphs, then at the end provide a table showing the comparison of the simulation metrics.

a)

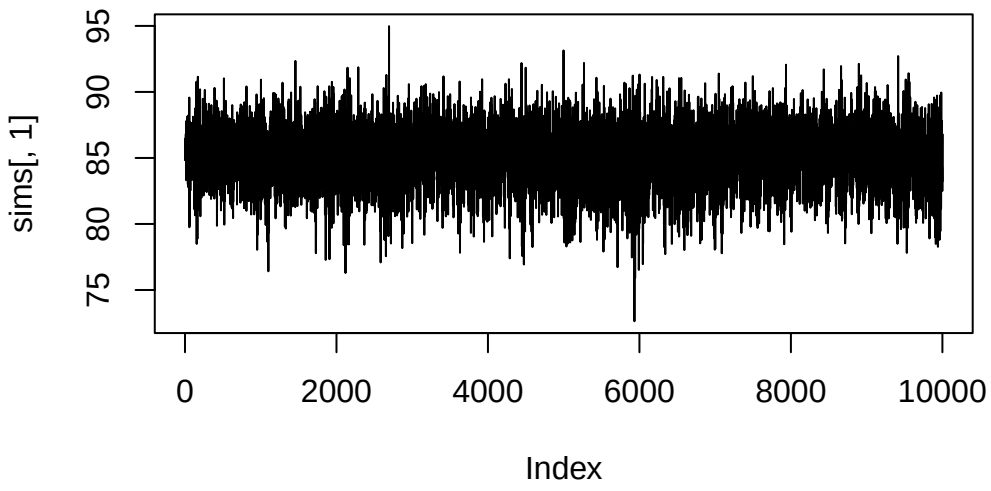
Given in class previously, the Conjugate Complete Conditional for μ is:

$f(\mu|\mathbf{y}, \sigma^2) \sim N(m1, v1), v1 = \frac{1}{\frac{1}{v} + \frac{n}{\sigma^2}}, m1 = v1\left(\frac{m}{v} + \frac{n\bar{x}}{\sigma^2}\right)$

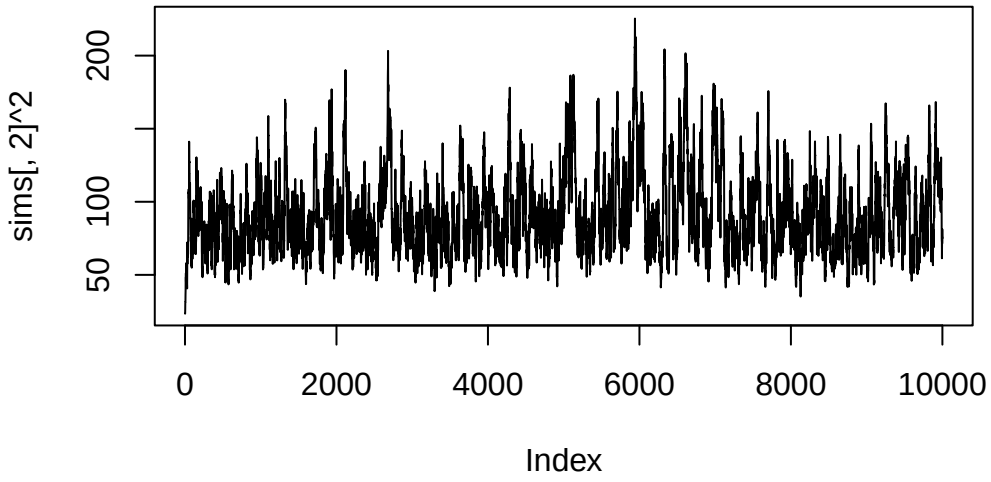
$p(\sigma|y, \mu) \propto (\sigma^2)^{-n/2} e^{\frac{-1}{2\sigma^2}\sum(y_i-\mu)^2} \frac{1}{s_0+\sigma^2}$

```
logfc_mu <- function(sig2, n, v, xbar) {
    v1 <- 1/(1/v + n/sig2)
    m1 <- v1*(m/v + n*xbar/sig2)
    rnorm(1, m1, v1)
}

logfc_sigma <- function(n, y, mu, sig2, s0) {
    -n/2 * log(sig2) - 1/(2*sig2)*(sum((y - mu)^2)) - log(s0 + sig2)
}

mh_sigma <- function(n, y, mu, sigma, s0) {
    proposal <- rnorm(1, mean = sigma, sd = .5)
    # Sigma is lower bounded by 0
    if (proposal <= 0) {return(sigma)}
    odds <- logfc_sigma(n, y, mu, proposal^2, s0) - logfc_sigma(n, y, mu, sigma^2, s0)
    ifelse(log(runif(1)) <= odds, proposal, sigma)
}

# Initialize
state <- list(mu = initial_mu, sigma = initial_sigma)
n_iter <- 10000
sims <- as.data.frame(matrix(NA, nrow = n_iter, ncol = length(state)))
colnames(sims) <- names(state)

start_time <- Sys.time()
for (i in 1:n_iter) {
    state$mu <- logfc_mu(state$sigma^2, n, v, ybar)
    state$sigma <- mh_sigma(n, y, state$mu, state$sigma, s0)

    # Save current state
    sims[i, 'mu'] <- state$mu
    sims[i, 'sigma'] <- state$sigma
}
end_time <- Sys.time()
run_time_a <- round(end_time - start_time, 4)
```

```r
plot(sims[, 1], type = 'l')
```




```r
plot(sims[, 2]^2, type = 'l')
```




```r
mu_post_mean_a <- round(mean(sims[, 1]), 4)
sigma2_post_mean_a <- round(mean(sims[, 2]^2), 4)
```

b)

$\pi(\sigma|\eta) \sim Inv-Gamma(\frac{1}{2}, \frac{1}{\eta}), \eta \sim Inv-Gamma(\frac{1}{2}, \frac{1}{s_0^2})$

To derive the posteriors/complete conditionals:

$p(y, \mu, \sigma) \propto \prod_{i=1}^{n} \left[(2\pi\sigma^2)^{-1/2} e^{\frac{-1}{2\sigma^2}(y_i-\mu)^2}\right] (2\pi v)^{-1/2} e^{\frac{-1}{2v^2}(\mu-m)^2} \frac{(\frac{1}{\eta})^{1/2}}{\Gamma(1/2)} (\sigma^2)^{-1/2-1} e^{\frac{-1}{\eta\sigma^2}} \frac{(\frac{1}{s_0})^{1/2}}{\Gamma(1/2)} \eta^{-1/2-1} e^{\frac{-1}{s_0^2\eta}}$

For σ^2:

$p(\sigma^2|\eta, \mu, y) \propto (\sigma^2)^{-n/2} e^{\frac{-1}{2\sigma^2}\sum(y_i-\mu)^2} (\sigma^2)^{-3/4} e^{\frac{-1}{\eta\sigma^2}} = (\sigma^2)^{-\frac{n+1}{2}} e^{\frac{-1}{\sigma^2}(\frac{\sum((y_i-\mu)^2)}{2} + \frac{1}{\eta})}$

This is an inverse gamma:

$p(\sigma^2|\eta,\mu,y) \sim Inv-Gamma(\frac{n+1}{2}, \frac{\sum((y_i-\mu)^2)}{2} + \frac{1}{\eta})$

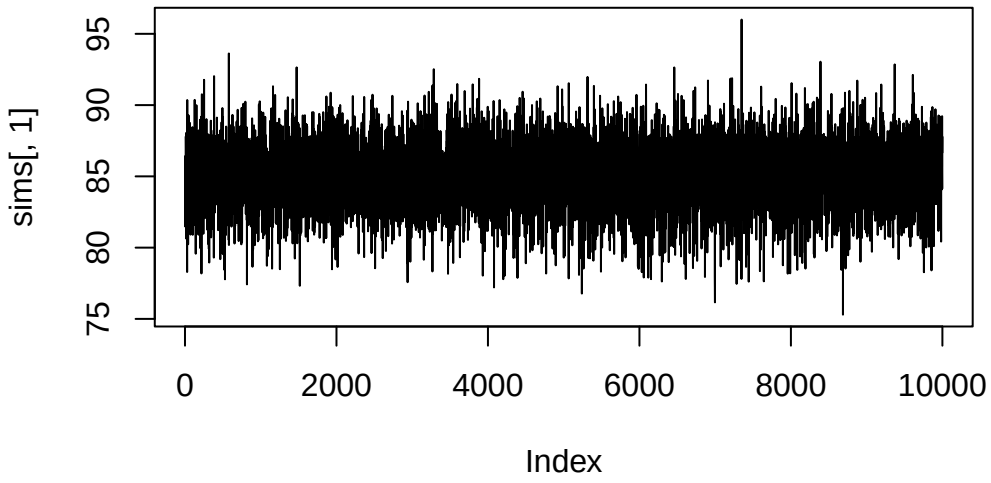
For η:

$p(\eta|\sigma^2,\mu,y) \propto (\eta)^{-1/2} e^{\frac{-1}{\sigma^2\eta}} \eta^{-3/2} e^{\frac{-1}{s_0^2\eta}} = \eta^{-1-1} e^{\frac{-1}{\eta}(\frac{1}{\sigma^2}+\frac{1}{s_0^2})}$

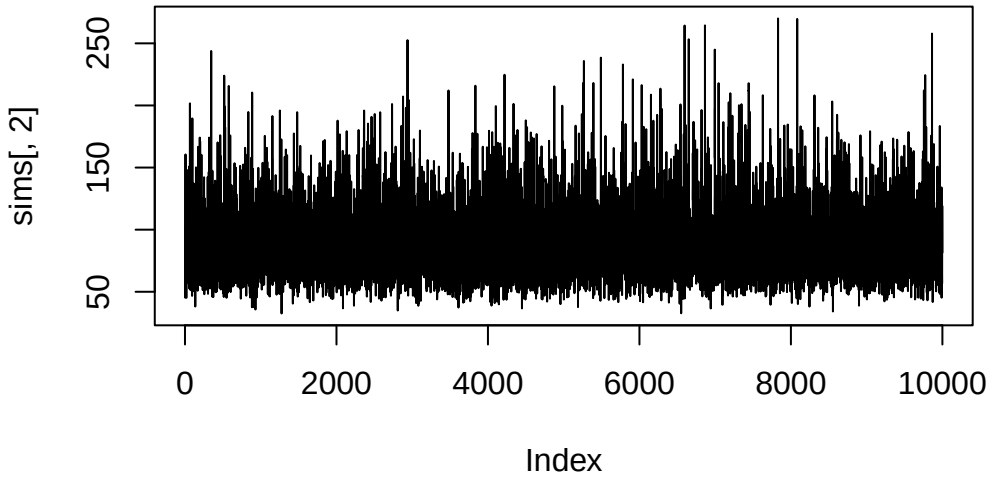
This is an inverse gamma:

$p(\eta|\sigma^2,\mu,y) \sim Inv-Gamma(1, \frac{1}{\sigma^2} + \frac{1}{s_0^2})$

```r
rinvgamma <- function(n, shape, scale) {
  1.0 / rgamma(n, shape = shape, rate = scale)
}

logfc_sigma <- function(n, y, mu, eta) {
    shape <- (n+1)/2
    scale <- sum((y-mu)^2)/2 + 1/eta
    rinvgamma(1, shape, scale)
}

logfc_eta <- function(sig2, s0) {
    rinvgamma(1, 1, 1/sig2 + 1/(s0^2))
}

# Initialize
state <- list(mu = initial_mu, sigma2 = initial_sigma^2, eta = 3)
n_iter <- 10000
sims <- as.data.frame(matrix(NA, nrow = n_iter, ncol = length(state)))
colnames(sims) <- names(state)

start_time <- Sys.time()
for (i in 1:n_iter) {
    state$mu <- logfc_mu(state$sigma2, n, v, ybar)
    state$sigma2 <- logfc_sigma(n, y, state$mu, state$eta)
    state$eta <- logfc_eta(state$sigma2, s0)

    # Save current state
    sims[i, 'mu'] <- state$mu
    sims[i, 'sigma2'] <- state$sigma
    sims[i, 'eta'] <- state$eta
}
end_time <- Sys.time()
run_time_b <- round(end_time - start_time, 4)

plot(sims[, 1], type = 'l')
```




```r
plot(sims[, 2], type = 'l')
```




```r
mu_post_mean_b <- round(mean(sims[, 1]), 4)
sigma2_post_mean_b <- round(mean(sims[, 2]), 4)
```

c)

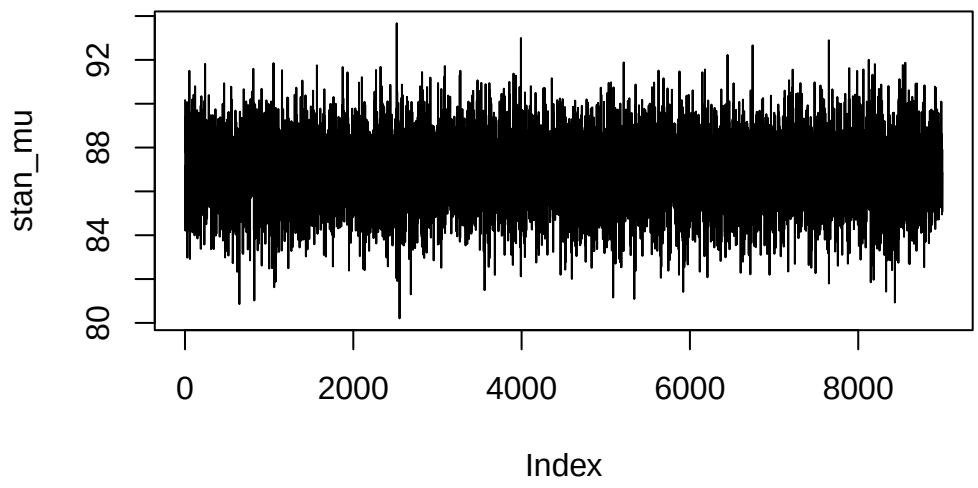
```
data {
    int<lower=0> N;
    array[N] int y;
    real m;
    real v;
    real s0;
}

parameters {
    real mu;
    real<lower=0> sigma;
```

```
}

model {
    y ~ normal(mu, sigma);
    mu ~ normal(m, v);
    sigma ~ cauchy(s0);
}
```

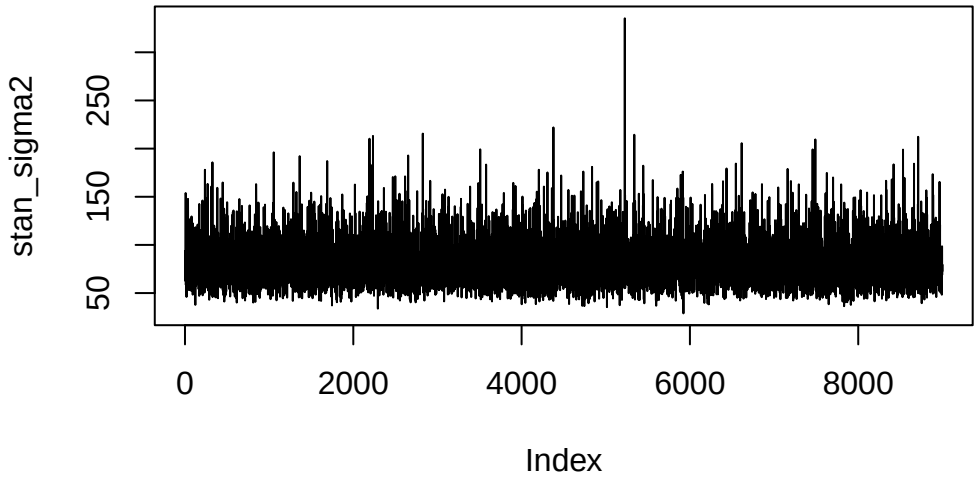
```
suppressPackageStartupMessages(library("rstan"))
start_time <- Sys.time()
#### STAN MODEL
data_stan <- list(N = n, y = y, m = m, v = v, s0 = s0)

params <- c("mu", "sigma")

n_cores <- parallel::detectCores()   # Count how many cores are available
options(mc.cores = n_cores)

fit_stan <- stan(model_code = readLines("q3.stan"),
                 data = data_stan, pars = params,
                 iter = 5000, warmup = 500, thin = 5, chains = n_cores)
```

```
Warning in readLines("q3.stan"): incomplete final line found on 'q3.stan'
```

```
sim_stan <- extract(fit_stan) # collect samples from all chains

stan_mu <- sim_stan$mu
stan_sigma2 <- sim_stan$sigma^2

end_time <- Sys.time()
run_time_c <- round(end_time - start_time, 4)

plot(stan_mu, type = 'l')
```



```r
plot(stan_sigma2, type = 'l')
```




```r
mu_post_mean_c <- round(mean(stan_mu), 4)
sigma2_post_mean_c <- round(mean(stan_sigma2), 4)
```

Part	CPU Time	Post. Mean. μ	Post. Mean σ^2
A	0.3262	85.0637	90.716
B	0.4578	85.0926	87.9331
C	14.5498	86.8813	81.4923

The posterior means for μ are pretty consistent, but we get a lot more variation on the posterior mean for σ^2. In particlar in the STAN sampling there is one point which is WAY high. STAN also takes the longest. So in general, I think if you can mathematically find some sort of conditioning, using full conditionals, etc., then it's much faster to do that. However, using STAN is WAY faster in terms of setup, and doesn't require finding posteriors, etc. It's more of just a plug and play which is nice. The other ones are fast, but require a decent investment beforehand in figuring out the math.

Acknolwedgements

Estimated time

I estimate this assignment took me in total around 6.5 hours to complete

Resources Used

I used online resources for programming help, particularly with the STAN code for cauchy, etc. I used lots of class resources, particularly the slides, in class code, and the solutions for HW4/HW5. Worked in TA lab on 2(a) and 2(b).